

Name of Student: Husam Nour

PRN:25070122086

Semester: III

Year : AY 2026-27

Subject: Programming with Java

ASSIGNMENT No : 6

TITLE : Inner and Anonymous Classes

Problem Statement

Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives

To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Programme

```

1  class Animal {
2      void eat(){
3          system.out.println("This animal eats food.");
4      }
5  }
6  class Dog extends Animal {
7      void bark() {
8          system.out.println("Dog barks!");
9      }
10 }
11 class cat extends Animal {
12     void meow(){
13         system.out.println("Cat meows!");
14     }
15 }
16 public class HierarchicInheritance {
17     public static void main(String[] args) {
18         Dog d = new Dog();
19         d.eat();
20         d.bark();
21
22         Cat c = new Cat();
23         c.eat();
24         c.meow();
25     }
26 }

```

```

1  class Grandparent {
2      void showGrandparent() {
3          System.out.println("Grandparent class method");
4      }
5  }
6  class parent extends Grandparent {
7      void showParent() {
8          System.out.println("Parent class method");
9      }
10 }
11 class child extends Parent {
12     void showChild() {
13         System.out.println("Child class method");
14     }
15 }
16 public class MultilevelInheritance {
17     public static void main(string[] args) {
18         Child c = new Child();
19         c.showGrandparent();
20         c.showParent();
21         c.shoeChild();
22     }
23 }

```

```
class Animal{
    void eat()
    {
        System.out.println("this animal eat food ");
    }
}

class Dog extends Animal

    void bark()
    {
        System.out.println(" dog barks");
    }

public class main

    public static void main (String [] args)
    {
        Dog d = new Dog();
        d.eat();
        d.bark();

    }
```

```
interface Animal
{
    void sound(); // abstract method
}

class Dog implements Animal
{
    public void sound()
    {
        System.out.println("dog barks ");
    }
}

class chat implements Animal
{
    public void sound()
    {
        System.out.println("chat crie ");
    }
}

class matou extends chat
```

```
{  
    public void sound()  
    {  
        System.out.println("matoubou  crie ");  
    }  
}
```

```
public class Interface  
{  
    public static void main(String[] args)  
    {  
        Animal a = new Dog();  
        Animal b = new chat();  
        chat m = new matou();  
        a.sound();  
        b.sound();  
        m.sound();  
    }  
}
```

```
class grandparent
{
    void showgrandparent()
    {
        System.out.println("grandparent class");
    }
}
class parent extends grandparent{
    void showparent()
    {
        System.out.println("parent class method");
    }
}
class child extends parent
{
    void showchild()
    {
        System.out.println("child class method");
    }
}
public class Grand
```

```

{
public class Grand
{
    public static void main(String [] args)
    {
        child c = new child();
        c.showgrandparent();
        c.showparent();
        c.showchild();

    }
}
```

```
class Animal
{
    void eat()
    {
        System.out.println("this animal eats food ");
    }
}
class dog extends Animal
{
    void bark()
    {
        System.out.println("dog barks");
    }
}
class cat extends Animal
{
    void miaou()
    {
        System.out.println(" le chat est beau");
    }
}
```

```
public class Hirarchical
{
    public static void main(String[] args)
    {
        dog d = new dog();
        cut c = new cut();

        d.bark();
        d.eat();

        c.miou();
        c.eat();
    }
}
```

Conclusion

The Java program demonstrates the use of an inner class and an anonymous class. The inner class shows how a class can be defined inside another class and access its members directly. The anonymous class demonstrates how to create and use a one-time implementation of a class or interface without giving it a name. These features help make Java programs more organized, flexible, and easier to maintain by reducing unnecessary code.